

Node.js

The Complete Guide

From First Principles to Production-Grade Applications

Basic · Intermediate · Advanced

A practical, example-driven reference covering the Node.js runtime, core modules, asynchronous programming, Express, databases, security, testing, and deployment.

Table of Contents

-
1. Introduction to Node.js
 2. Why Use Node.js?
 3. Node.js Architecture Explained
 4. Installing Node.js and Using NPM
 5. Modules in Node.js (CommonJS & ES Modules)
 6. Core Built-in Modules
 7. Asynchronous Programming
 8. The Event Loop In Depth
 9. Building an HTTP Server from Scratch
 10. Express.js Framework
 11. Middleware in Depth
 12. Building REST APIs
 13. Working with Databases (MongoDB)
 14. Error Handling Best Practices
 15. Streams and Buffers
 16. Child Processes, Cluster & Worker Threads
 17. Security Best Practices
 18. Testing Node.js Applications
 19. Deployment & Process Management
 20. Advanced Topics & Scaling
 21. Conclusion & Next Steps

1. Introduction to Node.js

What is Node.js?

Node.js is an open-source, cross-platform **JavaScript runtime environment** that allows developers to execute JavaScript code outside of a web browser. It was created by **Ryan Dahl in 2009** and is built on top of Google Chrome's **V8 JavaScript engine** — the same engine that powers Chrome's JavaScript execution.

Before Node.js existed, JavaScript could only run inside a browser. Node.js changed that by taking the V8 engine, embedding it inside a C++ program, and adding a set of APIs (file system access, networking, processes, etc.) that let JavaScript talk directly to the operating system. This meant JavaScript could now be used as a general-purpose, **server-side language**.

Key Characteristics

- **Single-threaded:** Node.js runs on a single main thread using an event loop, rather than spawning a new thread for every request.
- **Non-blocking / Asynchronous I/O:** Operations like reading files or querying a database don't block the thread — they run in the background and notify Node.js via callbacks when done.
- **Event-driven:** Node.js reacts to events (a request coming in, a file finishing reading) rather than executing code in a strict top-to-bottom blocking sequence.
- **Cross-platform:** Runs on Windows, macOS, and Linux with the same codebase.
- **NPM ecosystem:** Ships with npm, the largest software registry in the world, giving access to over a million open-source packages.

A Simple First Program

Here is the classic starting point — a one-line Node.js script:

```
// hello.js
console.log("Hello from Node.js!");

// Run it from your terminal:
// $ node hello.js
```

NOTE: Unlike browser JavaScript, a Node.js script has access to `process`, `require/import`, and the file system — but it does *not* have access to browser-only objects like `window` or `document`.

2. Why Use Node.js?

This is one of the most common interview questions for a Node.js developer, so it's worth understanding the reasoning deeply rather than memorizing a list.

2.1 One Language, Full Stack

With Node.js, teams can use **JavaScript on both the frontend and backend**. This reduces context-switching for developers, allows code and logic (like validation schemas) to be shared between client and server, and makes it easier for a single developer to work across the entire stack.

2.2 High Performance for I/O-Heavy Applications

Node.js excels at applications that spend most of their time waiting on I/O — network calls, database queries, file reads — rather than heavy CPU computation. Because I/O is non-blocking, a single Node.js process can handle thousands of concurrent connections without creating a new thread per connection, which keeps memory overhead low compared to traditional thread-per-request models.

2.3 Real-Time Applications

The event-driven architecture makes Node.js a natural fit for real-time applications such as chat apps, collaborative tools (like Google Docs-style editors), live dashboards, multiplayer games, and streaming services — especially when paired with WebSockets (e.g. Socket.io).

2.4 Massive Package Ecosystem (npm)

npm gives instant access to well over a million reusable packages, which drastically speeds up development. Need authentication? Use Passport.js. Need a web framework? Use Express or Fastify. Need to talk to MongoDB? Use Mongoose.

2.5 Strong Corporate & Community Backing

Node.js is used in production by companies such as Netflix, PayPal, LinkedIn, Uber, and Walmart, and is maintained by the OpenJS Foundation. This translates into long-term stability, frequent security updates, and a large hiring pool of experienced developers.

2.6 When NOT to Use Node.js

IMPORTANT: Node.js is not ideal for **CPU-intensive tasks** (heavy image/video processing, complex mathematical computation, machine learning training) because such tasks block the single main thread and stall the event loop. For these cases, consider offloading work to Worker Threads, a separate microservice in another language (e.g. Python, Go), or a job queue.

3. Node.js Architecture Explained

3.1 The Building Blocks

- **V8 Engine:** Google's high-performance JavaScript engine that compiles JS directly to machine code.
- **libuv:** A C library that provides the event loop, thread pool, and asynchronous I/O for file systems, networking, and DNS across all operating systems.
- **Node.js Bindings:** C++ bindings that connect JavaScript code to the underlying system APIs exposed by libuv.
- **Core Modules:** Built-in JavaScript modules (fs, http, path, etc.) written using the above bindings.

3.2 Single Thread, Multiple Helpers

Node.js executes your JavaScript on a **single main thread**. However, it is not truly limited to one thread overall — libuv maintains a background **thread pool** (default size 4) that handles expensive operations like file system access, DNS lookups, and some cryptography functions, so the main thread stays free to keep processing events.

3.3 Request Flow (Simplified)

```
Client Request
  |
  v
[ Event Loop (single thread) ]
  |
  |--- Simple task? -----> Handled immediately
  |
  |--- I/O task (file, DB, network)? -> Delegated to libuv
  |                                     (thread pool / OS async APIs)
  |                                     |
  |<----- callback queued <-----|
  v
[ Callback executed on next event loop tick ]
  |
  v
Response sent to Client
```

3.4 Why This Matters

Because the main thread never sits idle waiting for I/O, Node.js can accept and start processing a new incoming request while a previous request's database query is still running in the background. This is the core reason Node.js can serve high levels of concurrency with relatively low memory usage.

4. Installing Node.js and Using NPM

4.1 Installing Node.js

Node.js can be installed from the official website (nodejs.org), via a version manager, or via your OS package manager. Using a version manager is recommended for real projects since different projects often require different Node versions.

```
# Using nvm (Node Version Manager) - recommended
nvm install --lts
nvm use --lts

# Verify installation
node -v
npm -v
```

4.2 What is NPM?

npm (Node Package Manager) is installed automatically with Node.js. It is used to install third-party libraries, manage project dependencies, and run project scripts.

4.3 package.json

Every Node.js project has a `package.json` file describing the project, its dependencies, and runnable scripts.

```
npm init -y          # creates a default package.json

npm install express   # adds a dependency
npm install -D nodemon # adds a dev dependency
npm uninstall express  # removes a dependency
npm update             # updates dependencies
npm run <script-name> # runs a custom script
```

```
{
  "name": "my-app",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "jest"
  },
  "dependencies": {
    "express": "^4.19.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

4.4 Semantic Versioning

- **^4.19.2** — accepts new minor/patch versions (4.x.x), but not 5.0.0
- **~4.19.2** — accepts new patch versions only (4.19.x)
- **4.19.2** (no prefix) — locks to that exact version

5. Modules in Node.js

Node.js applications are organized into **modules** — reusable pieces of code kept in separate files. Node.js supports two module systems.

5.1 CommonJS (CJS) — the traditional system

```
// math.js
function add(a, b) {
  return a + b;
}
module.exports = { add };

// app.js
const { add } = require('./math');
console.log(add(2, 3)); // 5
```

5.2 ES Modules (ESM) — the modern standard

To use ES Modules, set "type": "module" in package.json, or use the .mjs extension.

```
// math.mjs
export function add(a, b) {
  return a + b;
}

// app.mjs
import { add } from './math.mjs';
console.log(add(2, 3)); // 5
```

5.3 CommonJS vs ES Modules

Feature	CommonJS	ES Modules
Import syntax	require()	import
Export syntax	module.exports	export
Loading	Synchronous	Asynchronous
Default in	Node.js (historically)	Browsers & modern Node.js
Top-level await	Not supported	Supported

6. Core Built-in Modules

Node.js ships with a rich standard library. Below are the modules used most often in real applications.

6.1 fs (File System)

```
const fs = require('fs');

// Asynchronous (non-blocking) - preferred
fs.readFile('data.txt', 'utf8', (err, data) => {
  if (err) throw err;
  console.log(data);
});

// Promise-based (modern, cleanest)
const fsp = require('fs/promises');
async function readData() {
  const data = await fsp.readFile('data.txt', 'utf8');
  console.log(data);
}
```

6.2 path

```
const path = require('path');

path.join('/users', 'sofi', 'file.txt'); // '/users/sofi/file.txt'
path.basename('/users/sofi/file.txt');   // 'file.txt'
path.extname('file.txt');                 // '.txt'
path.resolve('file.txt');                 // absolute path
```

6.3 http

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Hello World');
});

server.listen(3000, () => console.log('Server running on port 3000'));
```

6.4 os and process

```
const os = require('os');
console.log(os.platform()); // 'linux', 'darwin', 'win32'
console.log(os.cpus().length);

console.log(process.env.NODE_ENV);
console.log(process.argv); // command-line arguments
process.exit(0);
```

6.5 events

Node.js is built around the EventEmitter pattern, used internally by streams, HTTP, and more.

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

emitter.on('greet', (name) => console.log(`Hello, ${name}!`));
emitter.emit('greet', 'Sofi');
```

7. Asynchronous Programming

Asynchronous programming is the heart of Node.js. There are three main patterns, representing the evolution of how JavaScript has handled async code.

7.1 Callbacks (the original pattern)

```
fs.readFile('file.txt', 'utf8', (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});
```

Nesting many callbacks leads to **"callback hell"** — deeply indented, hard-to-read code.

7.2 Promises

```
const fsp = require('fs/promises');

fsp.readFile('file.txt', 'utf8')
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

7.3 async/await (modern standard)

```
async function loadFile() {
  try {
    const data = await fsp.readFile('file.txt', 'utf8');
    console.log(data);
  } catch (err) {
    console.error(err);
  }
}

loadFile();
```

7.4 Running Tasks in Parallel

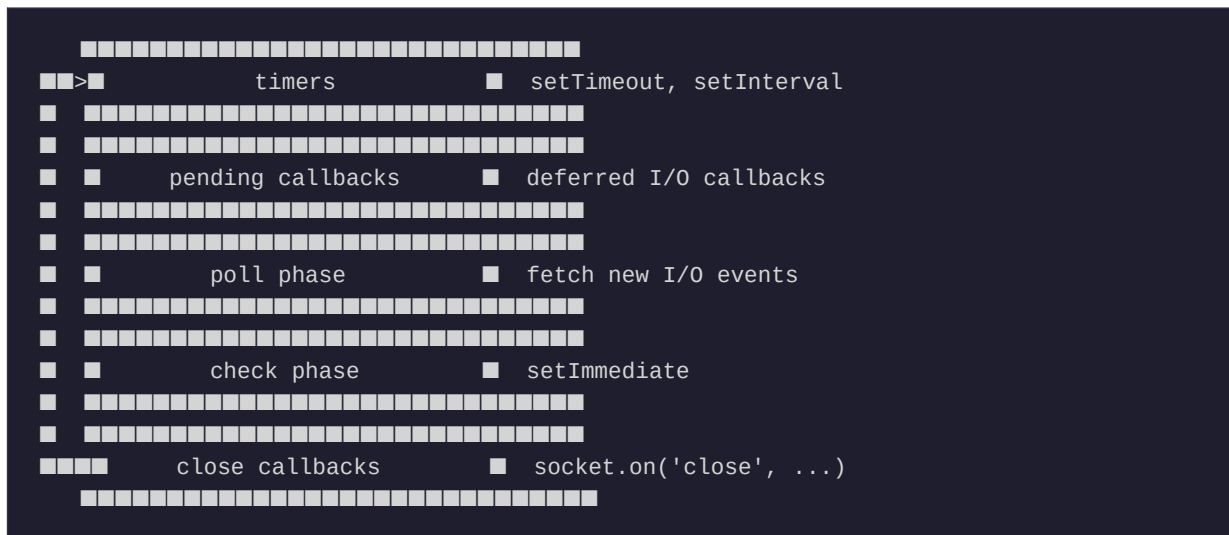
```
async function loadAll() {
  const [users, posts] = await Promise.all([
    fetch('/api/users').then(r => r.json()),
    fetch('/api/posts').then(r => r.json()),
  ]);
  console.log(users, posts);
}
```

NOTE: Use `Promise.all()` when tasks don't depend on each other — running them in parallel is much faster than awaiting them one by one in sequence.

8. The Event Loop In Depth

The event loop is what allows Node.js to perform non-blocking I/O despite JavaScript being single-threaded. It continuously checks whether there is work to do and processes it in well-defined phases.

8.1 Event Loop Phases



8.2 Microtasks vs Macrotasks

Between every phase of the event loop, Node.js drains the **microtask queue** (Promise callbacks, `process.nextTick`) before moving on. `process.nextTick()` callbacks run even before other microtasks.

```
console.log('1: start');

setTimeout(() => console.log('2: setTimeout'), 0);

Promise.resolve().then(() => console.log('3: promise'));

process.nextTick(() => console.log('4: nextTick'));

console.log('5: end');

// Output order: 1, 5, 4, 3, 2
```

8.3 Practical Takeaway

- Never run long, synchronous, CPU-heavy loops on the main thread — they block the event loop and freeze the entire server for every user.
- Prefer async APIs (fs/promises, database drivers with async support) over their synchronous counterparts (`readFileSync`) in server code.

- Use `process.nextTick` and `microtasks` sparingly — overusing them can starve the event loop of I/O processing time.

9. Building an HTTP Server from Scratch

Before reaching for a framework, it's valuable to understand how Node.js handles HTTP requests using only the built-in `http` module.

```
const http = require('http');
const url = require('url');

const server = http.createServer((req, res) => {
  const parsedUrl = url.parse(req.url, true);

  if (parsedUrl.pathname === '/' && req.method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ message: 'Welcome!' }));
  } else if (parsedUrl.pathname === '/users' && req.method === 'POST') {
    let body = '';
    req.on('data', chunk => (body += chunk));
    req.on('end', () => {
      const user = JSON.parse(body);
      res.writeHead(201, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ created: user }));
    });
  } else {
    res.writeHead(404, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Not Found' }));
  }
});

server.listen(3000, () => console.log('Listening on port 3000'));
```

NOTE: Manually parsing routes, methods, and request bodies like this works, but quickly becomes unmanageable at scale. This is exactly the problem frameworks like **Express.js** solve.

10. Express.js Framework

Express is the most popular Node.js web framework. It provides routing, middleware support, and a much cleaner API than the raw http module.

10.1 Basic Setup

```
npm install express

const express = require('express');
const app = express();

app.use(express.json()); // parse JSON request bodies

app.get('/', (req, res) => {
  res.json({ message: 'Welcome to Express!' });
});

app.post('/users', (req, res) => {
  const { name, email } = req.body;
  res.status(201).json({ id: 1, name, email });
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

10.2 Route Parameters & Query Strings

```
app.get('/users/:id', (req, res) => {
  const { id } = req.params;      // /users/42 -> id = '42'
  const { fields } = req.query;   // /users/42?fields=name -> fields = 'name'
  res.json({ id, fields });
});
```

10.3 Express Router (Modular Routes)

```
// routes/users.js
const router = require('express').Router();

router.get('/', (req, res) => res.json({ users: [] }));
router.get('/:id', (req, res) => res.json({ id: req.params.id }));

module.exports = router;

// app.js
const usersRouter = require('./routes/users');
app.use('/api/users', usersRouter);
```

11. Middleware in Depth

Middleware functions are functions that have access to the request object (req), the response object (res), and the next function in the application's request-response cycle. They form the backbone of Express.

11.1 Anatomy of Middleware

```
function logger(req, res, next) {  
  console.log(`${req.method} ${req.url} - ${new Date().toISOString()}`);  
  next(); // pass control to the next middleware/route  
}  
  
app.use(logger); // applied to every request
```

11.2 Types of Middleware

- **Application-level:** bound with `app.use()` or `app.METHOD()`, runs for every matching request.
- **Router-level:** same as application-level but bound to an `express.Router()` instance.
- **Error-handling:** takes four arguments (err, req, res, next) and handles thrown/passed errors.
- **Built-in:** `express.json()`, `express.static()`, `express.urlencoded()`.
- **Third-party:** `cors`, `helmet`, `morgan`, `express-rate-limit`, etc.

11.3 Error-Handling Middleware

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(err.status || 500).json({  
    error: err.message || 'Internal Server Error',  
  });  
});  
  
// Must be registered AFTER all other app.use()/routes
```

11.4 Common Third-Party Middleware

```
const helmet = require('helmet'); // security headers  
const cors = require('cors'); // enable CORS  
const morgan = require('morgan'); // request logging  
  
app.use(helmet());  
app.use(cors());  
app.use(morgan('dev'));
```

12. Building REST APIs

12.1 REST Principles

- **Resources** are represented as URLs (e.g. `/api/students`).
- **HTTP methods** represent actions: GET (read), POST (create), PUT/PATCH (update), DELETE (remove).
- **Statelessness**: each request contains all the information needed to process it — the server holds no client session state between requests.
- **Consistent response format** and correct status codes (200, 201, 400, 401, 404, 500, etc.).

12.2 A Full CRUD Example

```
const express = require('express');
const router = express.Router();
let students = [{ id: 1, name: 'Sofi', course: 'Full Stack Dev' }];

router.get('/', (req, res) => res.json(students));

router.get('/:id', (req, res) => {
  const student = students.find(s => s.id == req.params.id);
  if (!student) return res.status(404).json({ error: 'Not found' });
  res.json(student);
});

router.post('/', (req, res) => {
  const newStudent = { id: Date.now(), ...req.body };
  students.push(newStudent);
  res.status(201).json(newStudent);
});

router.put('/:id', (req, res) => {
  const idx = students.findIndex(s => s.id == req.params.id);
  if (idx === -1) return res.status(404).json({ error: 'Not found' });
  students[idx] = { ...students[idx], ...req.body };
  res.json(students[idx]);
});

router.delete('/:id', (req, res) => {
  students = students.filter(s => s.id != req.params.id);
  res.status(204).send();
});

module.exports = router;
```

12.3 Common HTTP Status Codes

Code	Meaning	Typical Use
200	OK	Successful GET/PUT

201	Created	Successful POST
204	No Content	Successful DELETE
400	Bad Request	Invalid input / validation error
401	Unauthorized	Missing/invalid authentication
403	Forbidden	Authenticated but not permitted
404	Not Found	Resource doesn't exist
500	Server Error	Unhandled exception

13. Working with Databases (MongoDB)

MongoDB, paired with the **Mongoose** ODM (Object Data Modeling library), is one of the most common database choices in the Node.js ecosystem due to its JSON-like document structure.

13.1 Connecting to MongoDB

```
npm install mongoose

const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/mydb')
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.error('Connection error:', err));
```

13.2 Defining a Schema and Model

```
const studentSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  course: { type: String, default: 'Undeclared' },
  createdAt: { type: Date, default: Date.now },
});

const Student = mongoose.model('Student', studentSchema);
```

13.3 CRUD with Mongoose

```
// Create
const student = await Student.create({ name: 'Sofi', email: 's@x.com' });

// Read
const all = await Student.find();
const one = await Student.findById(id);
const filtered = await Student.find({ course: 'MERN Stack' });

// Update
await Student.findByIdAndUpdate(id, { course: 'AI Full Stack' });

// Delete
await Student.findByIdAndDelete(id);
```

13.4 Relational Alternative: SQL with an ORM

For relational databases (MySQL, PostgreSQL), popular choices include **Sequelize** or **Prisma**, which provide schema modeling, migrations, and type-safe queries.

14. Error Handling Best Practices

14.1 Synchronous vs Asynchronous Errors

```
// Synchronous - use try/catch
try {
  JSON.parse('invalid json');
} catch (err) {
  console.error(err.message);
}

// Asynchronous - use try/catch with async/await
async function getData() {
  try {
    const data = await fetchFromDB();
    return data;
  } catch (err) {
    throw new Error('Failed to fetch data: ' + err.message);
  }
}
```

14.2 Centralized Error Handling in Express

```
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
    this.isOperational = true;
  }
}

app.get('/students/:id', async (req, res, next) => {
  try {
    const student = await Student.findById(req.params.id);
    if (!student) throw new AppError('Student not found', 404);
    res.json(student);
  } catch (err) {
    next(err); // forward to error-handling middleware
  }
});

app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  res.status(status).json({ error: err.message });
});
```

14.3 Handling Uncaught Errors Globally

```
process.on('uncaughtException', (err) => {
  console.error('Uncaught Exception:', err);
  process.exit(1); // restart process via PM2/cluster after cleanup
});

process.on('unhandledRejection', (reason) => {
  console.error('Unhandled Rejection:', reason);
});
```

NOTE: Never let 'uncaughtException' silently swallow errors and keep the process running — the application may be in a corrupted state. Log it, clean up, and exit; let a process manager restart it.

15. Streams and Buffers

15.1 What is a Buffer?

A **Buffer** is a fixed-size chunk of memory used to handle raw binary data (e.g. reading a file, receiving TCP data) — something regular JavaScript strings weren't designed for.

```
const buf = Buffer.from('Hello');
console.log(buf);           // <Buffer 48 65 6c 6c 6f>
console.log(buf.toString()); // 'Hello'
```

15.2 What is a Stream?

A **Stream** lets you process data piece-by-piece (in chunks) instead of loading an entire file or response into memory at once. This is critical for handling large files or network data efficiently.

Four Types of Streams

- **Readable** — data can be read from it (e.g. `fs.createReadStream`)
- **Writable** — data can be written to it (e.g. `fs.createWriteStream`)
- **Duplex** — both readable and writable (e.g. a TCP socket)
- **Transform** — a duplex stream that modifies data as it passes through (e.g. zlib compression)

15.3 Streaming a Large File

```
const fs = require('fs');

// Without streams: loads entire file into memory (bad for large files)
// const data = fs.readFileSync('huge-video.mp4');

// With streams: processes in small chunks (efficient)
const readStream = fs.createReadStream('huge-video.mp4');
const writeStream = fs.createWriteStream('copy.mp4');

readStream.pipe(writeStream);

readStream.on('end', () => console.log('Done copying!'));
readStream.on('error', (err) => console.error(err));
```

15.4 Streaming HTTP Responses

```
app.get('/download', (req, res) => {
  const fileStream = fs.createReadStream('report.pdf');
  res.setHeader('Content-Type', 'application/pdf');
  fileStream.pipe(res); // streams directly to the client
});
```

16. Child Processes, Cluster & Worker Threads

Since Node.js runs JavaScript on a single thread, these three tools exist to help applications take advantage of multi-core CPUs and offload heavy work.

16.1 `child_process`

Used to run external commands or scripts (including other languages) from within Node.js.

```
const { exec } = require('child_process');

exec('ls -la', (err, stdout, stderr) => {
  if (err) return console.error(err);
  console.log(stdout);
});
```

16.2 `cluster` module

Allows a Node.js application to spawn multiple worker processes (one per CPU core) that all share the same server port, improving throughput for CPU-bound workloads.

```
const cluster = require('cluster');
const os = require('os');

if (cluster.isPrimary) {
  const cpuCount = os.cpus().length;
  for (let i = 0; i < cpuCount; i++) cluster.fork();

  cluster.on('exit', (worker) => {
    console.log(`Worker ${worker.process.pid} died. Restarting...`);
    cluster.fork();
  });
} else {
  require('./app'); // each worker runs its own Express instance
}
```

16.3 `worker_threads`

Unlike `child_process` (separate OS processes) or `cluster` (multiple server instances), `worker_threads` run true parallel JavaScript threads within the same process — ideal for CPU-heavy tasks like image processing or complex calculations.

```
// worker.js
const { parentPort, workerData } = require('worker_threads');
const result = workerData.num * 2;
parentPort.postMessage(result);

// main.js
const { Worker } = require('worker_threads');
const worker = new Worker('./worker.js', { workerData: { num: 21 } });
worker.on('message', (result) => console.log(result)); // 42
```

17. Security Best Practices

17.1 Common Threats & Mitigations

- **Injection attacks (NoSQL/SQL):** always use parameterized queries / ODMs like Mongoose rather than concatenating raw user input into queries.
- **Cross-Site Scripting (XSS):** sanitize/escape user-generated content before rendering it; use the helmet middleware to set safe HTTP headers.
- **Cross-Site Request Forgery (CSRF):** use CSRF tokens for state-changing requests in cookie-based session apps.
- **Broken authentication:** hash passwords with bcrypt/argon2, never store plaintext passwords, enforce strong session/JWT expiry.
- **Sensitive data exposure:** never commit secrets to git — use environment variables (.env + dotenv).
- **Denial of Service:** apply rate limiting (express-rate-limit) and request size limits.

17.2 Password Hashing Example

```
const bcrypt = require('bcrypt');

async function registerUser(password) {
  const hashed = await bcrypt.hash(password, 10); // salt rounds = 10
  // store `hashed` in the database, never the raw password
}

async function login(password, hashedFromDB) {
  const match = await bcrypt.compare(password, hashedFromDB);
  return match; // true or false
}
```

17.3 JWT Authentication

```
const jwt = require('jsonwebtoken');

// Issue a token after successful login
const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET, {
  expiresIn: '1h',
});

// Middleware to protect routes
function authenticate(req, res, next) {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token provided' });

  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Invalid or expired token' });
  }
}
```

17.4 Environment Variables

```
npm install dotenv

// .env (never commit this file)
DB_URI=mongodb://localhost:27017/mydb
JWT_SECRET=super-secret-key

// app.js
require('dotenv').config();
console.log(process.env.DB_URI);
```

18. Testing Node.js Applications

18.1 Types of Tests

- **Unit tests** — test a single function or module in isolation.
- **Integration tests** — test how multiple parts work together (e.g. route + database).
- **End-to-end (E2E) tests** — simulate real user flows through the full running application.

18.2 Unit Testing with Jest

```
npm install -D jest

// sum.js
function sum(a, b) { return a + b; }
module.exports = sum;

// sum.test.js
const sum = require('./sum');

test('adds 2 + 3 to equal 5', () => {
  expect(sum(2, 3)).toBe(5);
});

// package.json -> "scripts": { "test": "jest" }
// Run with: npm test
```

18.3 Testing Express Routes with Supertest

```
npm install -D supertest

const request = require('supertest');
const app = require('./app');

test('GET /api/students returns 200', async () => {
  const res = await request(app).get('/api/students');
  expect(res.statusCode).toBe(200);
  expect(Array.isArray(res.body)).toBe(true);
});
```

18.4 Mocking

Mocking replaces real dependencies (databases, external APIs) with fake, controllable versions during tests, so tests run fast and don't depend on external systems being available.

```
jest.mock('./db');  
const db = require('./db');  
  
db.getUser.mockResolvedValue({ id: 1, name: 'Sofi' });
```

19. Deployment & Process Management

19.1 Preparing for Production

- Set `NODE_ENV=production` to enable framework-level optimizations.
- Never run a raw `'node app.js'` in production — use a process manager that restarts on crashes.
- Put the app behind a reverse proxy (Nginx) for TLS termination, compression, and load balancing.
- Log to a centralized service, not just the console, for production observability.

19.2 PM2 Process Manager

```
npm install -g pm2

pm2 start app.js --name my-api
pm2 start app.js -i max      # cluster mode across all CPU cores

pm2 list
pm2 logs my-api
pm2 restart my-api
pm2 stop my-api
pm2 startup                  # auto-start PM2 on server reboot
pm2 save
```

19.3 Dockerizing a Node.js App

```
# Dockerfile
FROM node:20-alpine

WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

COPY . .
EXPOSE 3000
CMD ["node", "app.js"]

docker build -t my-node-app .
docker run -p 3000:3000 my-node-app
```

19.4 Common Hosting Options

- **PaaS:** Render, Railway, Vercel (serverless), Heroku — fastest to get started.
- **VPS:** DigitalOcean, Linode, AWS EC2 — full control, requires manual setup (Nginx, PM2, TLS).
- **Containers:** AWS ECS/Fargate, Google Cloud Run, Kubernetes — for scalable, containerized deployments.

20. Advanced Topics & Scaling

20.1 Horizontal Scaling & Load Balancing

As traffic grows beyond what one server/process can handle, applications scale **horizontally** by running multiple instances behind a load balancer (Nginx, AWS ELB) that distributes incoming requests across them. Because Node.js instances should be **stateless**, session data typically moves to a shared store like Redis rather than in-memory.

20.2 Caching with Redis

```
const redis = require('redis');
const client = redis.createClient();
await client.connect();

app.get('/products', async (req, res) => {
  const cached = await client.get('products');
  if (cached) return res.json(JSON.parse(cached));

  const products = await Product.find();
  await client.setEx('products', 60, JSON.stringify(products)); // cache 60s
  res.json(products);
});
```

20.3 Microservices with Node.js

Instead of one large ('monolithic') application, large systems are often split into independent services (auth service, payments service, notifications service) that communicate over HTTP, message queues (RabbitMQ, Kafka), or gRPC. This allows teams to deploy, scale, and fail independently.

20.4 Performance Optimization Checklist

- Use compression middleware (gzip) to shrink response payloads.
- Use pagination and field selection for large database queries instead of returning entire collections.
- Add database indexes on frequently queried fields.
- Use connection pooling for database clients.
- Profile with tools like clinic.js or Node's built-in --prof flag to find real bottlenecks before optimizing blindly.
- Use a CDN for static assets so Node.js isn't serving them directly.

20.5 GraphQL as an Alternative to REST

Frameworks like Apollo Server let Node.js expose a single GraphQL endpoint where clients request exactly the fields they need, reducing over-fetching and under-fetching compared to fixed REST responses.

```
const { ApolloServer, gql } = require('apollo-server-express');

const typeDefs = gql`
  type Student { id: ID!, name: String!, course: String }
  type Query { students: [Student] }
`;

const resolvers = {
  Query: { students: () => Student.find() },
};
```

21. Conclusion & Next Steps

Node.js turns JavaScript into a powerful, general-purpose runtime capable of everything from simple scripts to large-scale, real-time, production systems. The concepts in this guide build on each other: understanding the **event loop** explains why asynchronous code matters; understanding **middleware** explains how Express is structured; and understanding **streams, clustering, and caching** explains how real applications scale in production.

Suggested Learning Path

- **Beginner:** core modules, async/await, building a basic Express CRUD API.
- **Intermediate:** authentication (JWT), MongoDB/Mongoose or PostgreSQL/Prisma, testing with Jest.
- **Advanced:** caching, worker threads, microservices, Docker, CI/CD, observability, and scaling patterns.

Recommended Practice Projects

- A REST API with authentication, role-based access, and file uploads.
- A real-time chat application using Socket.io.
- A RAG-style pipeline connecting Node.js to a vector database and an LLM.
- A microservice system with an API gateway routing to two or more independent services.

Prepared as a study reference — sofi-tech